TraceBack — Test Reports

Nov 6, 2026 · @Soumyaranjan

Summary

TraceBack passed all 474 functional tests (459 unit and integration, 15 system) and stopped all 23 attacks; both library audits are clean. The testing found 6 defects in the code and libraries, and all 6 are fixed and re-tested. Sign-in, access control and the claim rules held up under every attack.

TraceBack is the IIT Kanpur Lost & Found portal: a Next.js website, a FastAPI backend, a background worker and PostgreSQL. All testing ran against a copy of the code; the live site was not touched.

Level	What was tested	Tests	Passed	Failed / findings
Unit	Single functions and rules, in memory	113	113	0
Integration	API + database + files + email + AI queue	346	346	0
System	Full app in a real browser, 15 end-to-end scenarios	15	15	0
Adversarial	23 attacks + 2 dependency audits	25	23 attacks stopped, 2 audits clean	0 open (6 found, all fixed)

The 459 unit and integration tests together run 97.9% of the backend's lines and 91.2% of its branches, and each of its 57 files is above 90% (see the separate Code Coverage Report).

What the testing found and what was done.

- **Libraries:** Next.js 16.4.0 and Pillow 12.3.0 had published advisories; both were upgraded (16.4.5, 13.0.0) and the audits re-run clean.
- **Photo privacy:** phone photos kept their GPS location; metadata is now removed on upload, and the attack that found it is stopped.
- **Hardening:** security headers were missing and the API documentation was public; both are fixed on the website and the API.
- **Memory:** a 136 KB image could take 782 MB to decode; images over 60 megapixels are now refused before decoding.

The 270 tests added for Deliverable 3 (95 unit, 175 integration) and all the system, adversarial, concurrency and failure scripts are in the repository, so the team can re-run every level (see the Appendix).

1. Unit test report

Result: 113 of 113 unit tests passed (0 failed) in under 1 second. 18 already existed in the project; 95 were added for this report in `backend/tests/test_unit_logic.py`.

Scope. Business rules that can be checked on their own: security helpers, upload checks, email-domain rules, the claim state machine, item permissions, matching maths and input validation.

Approach. Each test builds users, items and claims in memory and never saves them. No database, API, email or network is involved, so a failure points to one function. Tool: pytest 9.1.1 on Python 3.14.6.

Area	Module under test	Tests	What is checked	Result
Claim state machine	`services/claims.py`	23	Only staff can make staff moves, and only the claimant can withdraw. Only the system can expire a claim. 8 impossible jumps are refused for everyone (for example submitted → collected, rejected → approved). Closed states have no exit.	23 passed
Input validation	`schemas/`	20	Text is tidied and blank optional fields become empty. Too-long or blank titles, descriptions and places are rejected, and so are bad dates. Phone numbers are stored as +91XXXXXXXXXX, and invalid ones are refused.	20 passed

Area	Module under test	Tests	What is checked	Result
Item permissions	`services/items.py`	16	Who can view, edit, close, cancel, change the drop-off office and add photos, at each status. ID-card photos are hidden from other students. The 3-photo limit and page counts are checked.	16 passed
Upload checks	`services/storage.py`	13	JPEG, PNG and WebP are recognised by their first bytes. GIF, a renamed .exe, PDF, SVG, RIFF audio and empty files are rejected. Folder names like `../app` are refused.	13 passed
ID-card reading (existing)	`services/ai/idcard.py`	10	Roll numbers are found despite common OCR mistakes. Only allowed email domains are used. Identifiers are masked for display.	10 passed
Matching maths	`services/ai/semantic.py`, `embeddings.py`	9	The date score fades over 30 days. The date phrase is computed, never guessed. Matching text holds no personal data. Vectors have unit length, similar texts score higher, and empty text gives no error.	9 passed
Email-domain rules	`services/auth.py`	8	Emails are trimmed and lowercased. Look-alike domains are refused (`evil-iitk.ac.in`, `iitk.ac.in.evil.com`, two `@` signs). Staff may use only @iitk.ac.in or @gmail.com.	8 passed

Area	Module under test	Tests	What is checked	Result
Security helpers	`core/security.py`	6	Sign-in codes are 6 digits and random. Session tokens are 256-bit and URL-safe. Hashes depend on the secret key, and only the right code matches its hash.	6 passed
Email rendering (existing)	`services/email.py`	5	The text and HTML versions are complete and HTML is escaped. The logo is embedded. SMTP without a password fails clearly. Sign-in codes are never logged.	5 passed
Other (existing)	agent, vision, search	3	When the AI agent is asked, parsing JSON answers, and building search queries.	3 passed

Coverage. Coverage is measured for the unit and integration tests together, because most files are exercised by both. The full suite of 459 tests runs 97.9% of the backend's lines and 91.2% of its branches; the Code Coverage Report gives the figures for every file.

Defects found: none in the code under test.

2. Integration test report

Result: 346 of 346 integration tests passed (0 failed); the full suite of 459 tests runs in about 26 seconds. 171 are the tests written during Deliverable 2 and 175 were added for Deliverable 3: for the command-line admin tool, the worker and seed script, the AI provider clients, email sending, the API's plumbing, the photo and capacity changes, the security fixes and failure recovery. They check that the API, the business rules, PostgreSQL, file storage, email and the AI queue work together.

Approach. Each test calls the real FastAPI app through an in-process HTTP client, against a real PostgreSQL 16 database. Each test runs inside a database transaction that is rolled back afterwards, so tests are independent and leave no data behind. External services are

replaced with fakes: emails are captured, a hashing embedder stands in for the AI model, and fake LLM, vision and OCR clients are used. Each client keeps its own cookies, so one test can act as several people.

Area	Test file	Tests	Main behaviours verified	Result
Items	`test_items.py`	33	Reporting lost and found items, validation, hidden-until-received, pagination, category filter, word-form search, photo upload and download, fake-image rejection, size and count limits, edit rules, the “I found this” flow, cancelling, changing the drop-off office	33 passed
Users and roles	`test_users_and_roles.py`	25	Onboarding and profile rules, unique roll numbers, required mobile number, the admin adding, promoting, demoting and removing staff, admins not being able to remove themselves, Gmail guard sign-in, removed guards being locked out	25 passed

Area	Test file	Tests	Main behaviours verified	Result
Claims	`test_claims.py`	21	Submitting, one open claim per person, private proof images, no skipped steps, the office-visit flow, reject and withdraw, approval email and deadline, hand-over only after the ID check, other claims closing, only one approved claim (checked by the database), expiry	21 passed
AI agent	`test_ai_agent.py`	14	Ranking with its own reasons, owner emailed once, weak matches never emailed, IDs it wasn't shown refused, fallback when the provider fails, daily AI-call limit, "off" and "ambiguous" modes, AI date claims replaced by the real date difference	14 passed
Staff desk	`test_staff.py`	13	Students blocked, receiving an item (who, where, when), walk-ins, transfers between offices, dispose and donate, history visible only to staff	13 passed

Area	Test file	Tests	Main behaviours verified	Result
Sign-in	`test_auth.py`	12	Domain check, full sign-in, codes stored only as hashes, single use, lock after wrong attempts, expiry, resend wait, hourly limit, 401 when signed out, logout, deactivated users	12 passed
ID-card owner alerts	`test_ai_idcard.py`	12	Owner emailed once the card is at an office, the matching student account linked, the finder never emailed, the agent never sees identifiers, photos shown only to the finder and staff	12 passed
Background jobs	`test_jobs.py`	12	Drop-off, collection and review reminders sent once each, 7-day expiry, proofs deleted 2 days after closing, Returned page, 60-day purge, one failing job not stopping the others	12 passed

Area	Test file	Tests	Main behaviours verified	Result
Semantic matching	`test_ai_matching.py`	11	Matches stored in both directions, unrelated items not suggested, top-k limit, found-before-lost excluded, one job per save, re-matching after edits, embedder failure handled	11 passed
Similar-items panel	`test_similar.py`	9	Only the reporter and staff see suggestions, best-first order, hidden items dropped, scores clamped, a broken matcher never breaks the page	9 passed
AI photo descriptions	`test_ai_vision.py`	7	A description is shown on the item page and helps matching, ID-card photos are never sent to the AI, a vision failure doesn't block matching	7 passed
Email delivery	`test_email.py`	2	Sign-in reports when an email can't be sent, and every attempt is logged	2 passed

Area	Test file	Tests	Main behaviours verified	Result
Admin command-line tool	`test_cli.py`	53	make-admin (new and existing users, other domains refused), demo items, AI re-indexing of active reports only, the email log and its filters, item-check following an ID card through each stage, email-test, ai-check per provider, ai-run, ocr-check on a photo or an item, and every command's errors for missing items or files	53 passed
AI provider clients	`test_ai_clients.py`	41	Groq first and Gemini as fallback, keys without a model ignored, AI switched off, the daily AI-call count, the photo-description client, the embedder choice and its fallback to hashing, the OCR engine and ID-card scanning, match reasons, jobs for deleted items skipped (against a fake SDK)	41 passed

Area	Test file	Tests	Main behaviours verified	Result
Worker and seed script	`test_worker_and_seed.py`	14	The seed adds only what is missing; the worker runs every timed job, sends a due reminder once, keeps its schedule, survives AI errors and stops on Ctrl+C; the AI runner works through its queue and recovers from a failed round	14 passed
API plumbing and edge cases	`test_api_edges.py`	13	Health check (200, or 503 without a database), a database session per request that is always closed, a pool large enough for every request in flight, the AI runner started and stopped with the API, proofs opened only through their own claim, field tidying, the office list	13 passed

Area	Test file	Tests	Main behaviours verified	Result
Email sending	`test_email_delivery.py`	12	TLS before the password over SMTP, the From address, mail-server failures reported and logged, the email log (never showing sign-in codes, long values cut to fit, never stopping an email), the console preview	12 passed
Photo and capacity changes	`test_performance_changes.py`	7	Small WebP thumbnails without EXIF and with the same permission rules, thumbnails made for older photos, photos over 60 megapixels refused, one query for session and user, requests over the limit wait and then get 503	7 passed
Security fixes	`test_hardening.py`	25	Photo metadata removed on upload (GPS gone, picture and rotation kept, damaged files refused), security headers on every API response, API docs switch, email retries, no placeholder office text in emails, agent settings	25 passed

Area	Test file	Tests	Main behaviours verified	Result
Failure recovery	`test_failure_recovery.py`	10	Clear 503 while the database is unreachable, interrupted AI jobs run again (up to 3 tries), a report and its AI job saved in one commit, an unsent sign-in code doesn't block a retry	10 passed

Coverage. With the unit tests, the suite runs 97.9% of the backend (3,445 of 3,520 lines) and 91.2% of its branches. By area: core 100%, routers 99.7%, schemas 99.0%, models 98.9%, AI services 98.0%, services 97.2%, and the entry points (command-line tool, worker, seed script) 95.8%. The real AI providers and the downloaded embedding model are replaced with fakes, but their client code is tested against a fake SDK that answers as Groq or Gemini would.

Defects found: none.

3. System test report

Result: 15 of 15 system tests passed on the final run. They follow a complete lost-and-found journey through a real web browser, on the whole app running as it does on the server. On the first run 2 tests failed because of mistakes in the test script, not the app (explained below the table). Both were corrected and the full suite was run again on a fresh database.

Setup. The Next.js production build (`next start`) proxies `/api` to FastAPI (uvicorn). The background worker runs as its own process. The database is a fresh PostgreSQL 16 with migrations and starting data, and the admin was created with `make-admin` . A headless Chromium browser (Playwright) uses four separate sessions as four people: a student who lost an item, a student who found it, the admin and a guard. Emails ran in console mode, so the tests read each sign-in code from the backend log, as a user reads their inbox.

ID	Scenario	Expected result	Actual result	Result
SYS-01	Health check through the website, then open a private page while signed out	Health reports OK; private page sends you to sign-in	`{"status":"ok","database":"ok"}`; `/my-reports` redirected to `/login`	Pass
SYS-02	Student signs in with an emailed code and completes onboarding	Code accepted; profile saved; home page shown	As expected	Pass
SYS-03	Student reports a lost calculator through the form	Report created and its page opens	Report LST-0001 created	Pass
SYS-04	Second student reports the found calculator with a photo	Report saved with photo; hidden from the Found page until handed in	Saved with photo; not listed for other students	Pass
SYS-05	Admin signs in as staff, opens the staff desk and receives the item	Item listed as waiting; status becomes held at the chosen office	Received at Main Security Office	Pass
SYS-06	Owner browses and searches Found items, then opens their lost report	Item listed; “casio” finds it, “umbrella” doesn’t; AI suggests it as a match	Shown as a strong match with reasons; the worker also emailed the owner	Pass
SYS-07	Owner claims the item	Claim sent and claim page shown	Claim CLM-0001 sent	Pass

ID	Scenario	Expected result	Actual result	Result
SYS-08	Staff starts the review and approves	Status Approved; owner emailed	Approved; approval email printed	Pass
SYS-09	Staff hands the item over	Not possible until the ID-card check is ticked; then shown on the Returned page	Button disabled until ticked; then handed over and listed on Returned	Pass
SYS-10	Admin adds a guard on Manage staff; the guard signs in	Guard can sign in and use the staff desk	As expected	Pass
SYS-11	Student opens `/staff`, then signs out	Staff desk not shown; after sign-out, private pages go to sign-in	As expected	Pass
SYS-12	Background worker	Starts and runs its jobs without errors	Running, no errors; sent the match email	Pass
SYS-13	6 main pages on a 390 px phone screen	No sideways scrolling	All 6 pages fit	Pass
SYS-14	Browser console on 6 main pages	No errors	0 console errors, 0 uncaught exceptions	Pass
SYS-15	Response times under light load	Page under 3 s, API under 500 ms (95th percentile)	Pages about 0.6 s (median); API median 5 ms, 95th percentile 7 ms over 50 requests	Pass

Defects found: none. Every page fit a 390 px phone screen and no page logged a browser error.

Outside services. Real email delivery and the real AI providers depend on outside services, so these tests use the console mailer and the small built-in embedder. Delivery was checked on the live site: on 4 Nov 2026 a sign-in email reached both a Gmail and an @iitk.ac.in mailbox. What happens when the mail server or the AI providers fail is tested in the Failure and Recovery report (FR-07 to FR-09).

4. Adversarial test report

Result: all 23 attacks were stopped, and both library audits are clean. No attack signed in as someone else, opened another student's private reports, claims or proofs, or gained staff powers. The four attacks that first got through (ADV-16, 17, 19, 21) and the two audits that first flagged old libraries (ADV-24, 25) led to fixes; the table shows the final run.

Approach. A script (`adversarial_tests.py`) attacked the running copy from section 3 as a signed-in student ("attacker") and as an outsider. A second student ("victim") owned the data under attack: a hidden found report, an ID-card item with a photo, a lost report, and a claim with a proof image. Attacks went straight to the API, skipping the website's checks, as a real attacker would. The XSS check opened the poisoned page in a real browser. Libraries were checked with `npm audit` and `pip-audit`. The live site was not attacked.

ID	Attack	Expected	Observed	Result
ADV-01	Guess the sign-in code: 5 wrong tries, then the right one	Code locked after 5 tries	All 5 refused; the correct code was also refused afterwards	Stopped
ADV-02	Flood a mailbox with code requests	Refused within 60 s	202, then 429	Stopped
ADV-03	Sign in with look-alike domains, a Cyrillic i, an unknown Gmail, or a newline + `Bcc:` header injection	All refused	4 × 400, 3 × 422; no email sent	Stopped
ADV-04	Made-up or tampered session cookie	401	401 for both	Stopped
ADV-05	Inspect the cookie; reuse it after sign-out	HttpOnly and SameSite set; dead after logout	HttpOnly, SameSite=Lax, Path=/; 401 after logout	Stopped

ID	Attack	Expected	Observed	Result
ADV-06	Student calls 5 staff endpoints directly (receive, review, approve, dispose)	403	403 for all 5	Stopped
ADV-07	Student calls admin endpoints to make themselves admin	403	403	Stopped
ADV-08	Profile update carrying `role=ADMIN`, a new email, name and roll number	Ignored	All unchanged	Stopped
ADV-09	IDOR: change ids in the URL to read, edit, close, delete or withdraw the victim's items, claim, proof image and ID-card photo (9 attempts)	Refused	403 or 404 for all 9; victim's data unchanged	Stopped
ADV-10	New report with extra fields: `status`, `kind`, `reporter_id`, `id`, `ai_description`	Ignored	Saved as an open lost report under a new id	Stopped
ADV-11	Claim your own find; claim one item twice	Both refused	400; 201 then 409	Stopped
ADV-12	SQL injection in search: `OR 1=1`, `DROP TABLE`, `UNION SELECT`, `pg_sleep`	No extra rows, no delay	0 rows, at most 0.08 s; tables intact	Stopped
ADV-13	Injection or abuse through `kind`, `category`, the id and page size (1,000,000)	Validation errors	422, or 0 results	Stopped
ADV-14	Stored XSS in an item title and description, viewed by another student	Shown as text; nothing runs	Shown as text; no script ran in Chromium	Stopped

ID	Attack	Expected	Observed	Result
ADV-15	Upload a PHP shell named .jpg, an SVG with script, a renamed .exe, HTML, an empty file and a 6 MB file	All rejected	400 for all 6	Stopped
ADV-16	Polyglot (JPEG header + script) named `../../app/main.py`	Stored harmlessly; served with `nosniff`	Stored under a random name, served as image/jpeg with `X-Content-Type-Options: nosniff`	Stopped
ADV-17	Phone photo with GPS location, downloaded by another student	Location removed	No GPS data in the downloaded photo; the picture itself is unchanged	Stopped
ADV-18	Decompression bomb: a 136 KB PNG of 12,000 × 12,000 pixels	Refused, or memory bounded	Refused with 400: over the 60-megapixel limit, so it never reaches the AI photo step	Stopped
ADV-19	Website security headers	CSP or frame protection, nosniff, Referrer-Policy	All 4 set; no X-Powered-By	Stopped
ADV-20	Cross-origin request from another website	Not allowed	No CORS permission given	Stopped
ADV-21	Public API documentation	Off in production	`/docs`, `/redoc` and `/openapi.json` all return 404	Stopped
ADV-22	Malformed JSON, path traversal and wrong types to provoke errors	No internals revealed	422 / 404; no stack traces	Stopped
ADV-23	30 rapid code guesses against a real account	No sign-in	All refused (400)	Stopped
ADV-24	`npm audit` on the website's libraries	No known vulnerabilities	Next.js 16.4.5: found 0 vulnerabilities	Clean

ID	Attack	Expected	Observed	Result
ADV-25	`pip-audit` on the backend's libraries	No known vulnerabilities	Pillow 13.0.0 and every other backend library: no known vulnerabilities found	Clean

Notes on the findings.

- ADV-24 and ADV-25 first flagged Next.js 16.4.0 (3 advisories, 2 of them critical) and Pillow 12.3.0 (39 advisories). Both were upgraded and the audits were run again on the final code.
- ADV-16, ADV-17, ADV-19 and ADV-21 first found a missing nosniff header, GPS data in shared photos, missing security headers and public API documentation. All four were fixed and the attacks run again on the final code.
- On the live site, Caddy forwards everything under `/api` to the backend, so the same checks apply at `/api/docs` and `/api/openapi.json` once the release is deployed.

5. Defects and recommendations

The four test levels found 6 defects, and all 6 are fixed and re-tested. IDs are those of the project's defect register (Defects Found and Fixed tab), which also lists the defects found by the concurrency, load and failure tests. No defect broke authentication, access control or the claim rules.

ID	Severity	Defect	Found by	Fix	Re-test
D-11	High	Next.js 16.4.0 had 3 published advisories (2 critical)	ADV-24	Upgraded to Next.js 16.4.5	`npm audit`: 0 vulnerabilities
D-12	Medium	Pillow 12.3.0, which decodes every uploaded photo, had 39 published advisories	ADV-25	Upgraded to Pillow 13.0.0 (with fastembed 0.9.1)	`pip-audit`: none found
D-13	Medium	Photos kept their EXIF data, including the GPS location, and other students could download them	ADV-17	Metadata removed on upload; JPEGs cleaned without re-encoding, only the rotation flag kept	ADV-17 stopped; `test_hardening.py`
D-08	Medium	A 136 KB PNG of 12,000 × 12,000 pixels needed 782 MB of memory to decode	ADV-18	Images over 60 megapixels refused before decoding	ADV-18 stopped (400)

ID	Severity	Defect	Found by	Fix	Re-test
D-14	Low	No security headers on the website or photo downloads; the site advertised X-Powered-By: Next.js	ADV-16, ADV-19	Headers set by Next.js, FastAPI and Caddy; X-Powered-By off	ADV-16, ADV-19 stopped
D-15	Low	The API documentation (/docs, /redoc, /openapi.json) was public	ADV-21	Off in production (`API_DOCS=false`)	ADV-21 stopped (404)

Observations (not defects).

- Code guessing has no per-IP limit. Each code allows only 5 tries and lasts 10 minutes, so guessing stays impractical (ADV-01, ADV-23). A per-IP limit in Caddy would add a second layer.
- The parts that use paid or external services were replaced with fakes in automated tests: real SMTP, Groq, Gemini and the downloaded embedding model. Keep checking them by hand before each demo.
- Tests ran on PostgreSQL 16, while production runs 17. A difference is unlikely to matter, but one run on 17 would confirm it.

Next steps.

1. Run every level before each deployment: `pytest`, then the system, adversarial, concurrency and failure scripts against a fresh copy.
2. After deploying, repeat the header and API-docs checks (ADV-19, ADV-21) against the live address.
3. Add component tests for the Next.js pages, which today are covered by the browser tests only.

Appendix: test environment and how to re-run

All tests ran on 6 Nov 2026 on a copy of the project's code, in an isolated Linux environment. The live site at 16-176-186-149.sslip.io and its data were not touched.

Component	Version used in testing	Production
Python	3.14.6	3.14 (Docker `python:3.14-slim`)

Component	Version used in testing	Production
FastAPI / SQLAlchemy / pytest	0.143.0 / 2.1.4 / 9.1.1	same (pinned in `requirements.txt`)
PostgreSQL	16	17
Next.js / React	16.4.5 / 19.3.0, production build	same
Browser	Chromium (headless, Playwright)	users' browsers
Email	console mode (codes read from the log)	Gmail SMTP
Embeddings for AI matching	hashing embedder	BAAI/bge-small-en-v1.5 model

Where the tests are (in the project folder).

- `backend/tests/`: 21 pytest files with 459 unit and integration tests.
- `testing/system_tests.py`: the 15 system tests.
- `testing/adversarial_tests.py` and `xss_check.py`: the 23 attacks.
- `testing/concurrency_tests.py` and `testing/failure_recovery_tests.py` (with `fake_smtp.py`): the concurrency and failure suites; `testing/perf/`: the load tests.
- `testing/start_stack.sh`: starts the full app on a fresh test database.
- `testing/results/`: the evidence of the final run: JSON results and run logs, 10 screenshots, the pytest JUnit report and the line-by-line coverage report (open `coverage_html/index.html`).

How to re-run.

1. Unit and integration: `cd backend` then `pytest` (about 26 s). Add `--cov=app --cov-branch` for coverage.
2. System: `bash testing/start_stack.sh`, then `python testing/system_tests.py`.
3. Adversarial: `bash testing/start_stack.sh` again, then `python testing/adversarial_tests.py`.
4. Concurrency: `bash testing/start_stack.sh` again, then `python testing/concurrency_tests.py`.
5. Failure and recovery: `python testing/failure_recovery_tests.py` (it starts its own copy; see `testing/README.md`).

Steps 2 and 3 need Linux, macOS or WSL, a built frontend and Playwright. `testing/README.md` lists the one-time setup. `start_stack.sh` drops and recreates its own test database, so never point it at real data.